



24BCF10059 - Lakshdeep

Experiment 3

Aim :

To study and implement advanced SQL concepts such as correlated aggregate subqueries, the HAVING clause, and the DENSE RANK() window function to solve real-world data analysis and reporting problems.

Question 1

You are given the Employee schema shown below; the task is to find the highest score for each respective department. For example, department D1 has the highest score as 5.28; hence in the output, the final updated score for every row belonging to D1 should be 5.28. The same logic applies to the other departments, i.e., D2, D3 and D4.

Solution :

```
UPDATE Employee AS E
SET SCORES = (
    SELECT MAX(SCORES)
    FROM Employee
    WHERE DEPT = E.DEPT
);

SELECT * FROM Employee;
```

Question 2

A company maintains records of products sold by different salespersons. The management wants to prepare a sales performance report by identifying the salesperson(s) who generated the highest overall sales revenue. If multiple salespersons have the same highest total sales amount, all of them must be included in the final report. Return the seller ID(s) satisfying this condition.

Solution :

```
SELECT seller_id
FROM Orders
GROUP BY seller_id
HAVING SUM(amount) =
(
    SELECT MAX(Profit)
    FROM(
        SELECT seller_id , SUM(amount) AS Profit
        FROM Orders
        GROUP BY seller_id
    )
);
```

Question 3

A company maintains employee salary records for different departments. The management wants to prepare a report containing employees whose salaries fall among the top three highest distinct salary values within their respective departments. If multiple employees receive the same qualifying salary, all such employees must be included in the final report. Display the department name, employee name, and salary for every qualifying employee

Solution :

```
SELECT D.name , E1.name , E1.salary
FROM Employee AS E1
JOIN
Department AS D
ON E1.departmentID = D.id
WHERE (
    SELECT COUNT(DISTINCT E2.salary)
    FROM Employee AS E2
    WHERE E2.departmentID = E1.departmentID
    AND
    E2.salary > E1.salary
) < 3
ORDER BY D.name
```

Learning Outcomes :

After completing this experiment, the following learning outcomes were achieved.

- Understood how to design reporting queries using GROUP BY together with correlated aggregate subqueries. Implemented aggregate functions and GROUP BY operations.
- Applied UPDATE ... FROM to modify records based on grouped aggregate values. Applied INNER JOIN and SELF JOIN for relational data retrieval.
- Implemented the DENSE RANK() window function to rank salaries within a partition. Generated unique combinations using self joins.
- Learned to filter ranked results using a nested subquery to retrieve top-N distinct values per group.